ExpenseMate - Final Project Report

Version 1.0. Prepared by: Syed Arham (24K-2551) and Muhammad Huzaifa (24K-3034).

ExpenseMate is a personal expense manager built from scratch as a Flutter client, a Node.js REST API and a MySQL database, following the Requirements, Design, Construction, Testing and Maintenance phases.

1. Project Information and Introduction

1.1 Project Information

Item	Detail
Project title	ExpenseMate - Personal Expense Manager
Version	1.0.0
Course	Software Construction
Team Lead (submitting)	Syed Arham - 24K-2551 - MemberA
Team Member	Muhammad Huzaifa - 24K-3034 - MemberB
Technology	Flutter 3.27.4 (Dart), Node.js 18 with Express 4, MySQL 8.0

1.2 Problem Statement

People often lose track of where their money goes because income, expenses and budgets are kept in notes or memory. A personal finance tool must record transactions quickly, warn when spending approaches a limit, and show where money is spent.

1.3 Objectives

- Record income and expenses with categories, accounts, dates and receipts.
- Set monthly budgets per category and warn through colour-coded progress as limits are approached.
- Provide category-wise analytics over day, week, month and year periods.

- Export transactions as CSV or PDF.
- Apply the Software Construction process from requirements to testing and maintenance.

1.4 Scope

Version 1.0 covers authentication with email OTP, transactions, accounts, categories, budgets, analytics, export, profile settings and currency conversion. Cloud backup and CSV import are not part of this version.

2. Phase 1: Problem Definition and Requirements

The full requirements are in the separate SRS document: 22 functional requirements (FR-01 to FR-22) and 10 non-functional requirements (NFR-01 to NFR-10). Member A wrote the requirements and Member B drew the use case diagram.

2.1 Functional Requirements Summary

Area	Main requirements
Authentication	Registration, email OTP verification (10-minute expiry, 5 attempts), login, forgot and reset password
Transactions	Add income and expense, receipt attachment, search and filters, delete
Accounts and categories	Cash, bank, card and wallet accounts with computed balances; 13 default categories; custom categories
Budgets	Monthly limit per expense category, carry-forward, green, amber and red alerts at 85 and 100 percent
Analytics	Dashboard overview; day, week, month and year trends; category breakdown
Export and settings	CSV and PDF export by email; profile, password, currency conversion, theme colour and dark mode

2.2 Non-Functional Requirements Summary

Category	Requirement
Security	bcrypt password hashes, SHA-256 OTP hashes, JWT with 7-day expiry
Performance	Cached data shown immediately; list requests within 2 seconds locally
Usability and portability	Material 3 interface on Android, iOS and web from one code base
Reliability and integrity	Currency conversion in a single transaction; foreign keys and unique constraints in the database
Maintainability	Layered backend with centralised validation and one error format

2.3 Use Case Diagram

Insert Figure 1 here: Use Case Diagram (diagram D1). The actor is the User; the SMTP email server is a secondary actor for OTP and export emails. The diagram groups 24 use cases into authentication, transactions, budgeting, analytics, data management and profile settings.

3. Phase 2: Architecture and Construction Planning

3.1 Architectural Style

The team chose Client-Server (local) for the whole system and a Layered style inside the server. The Flutter client talks to the Node.js API only through REST and JSON, and the API is the only component that touches the database.

Reason	Explanation
Separation of concerns	UI, business logic and data storage change independently

Reason	Explanation
Security	Passwords, JWT checks and validation stay on the server, never on the device
Reuse	The same API can serve Android, iOS and web clients
Testability	Each layer (routes, controllers, services, utilities) can be tested alone
Local deployment	Server and database can run on one machine during development

The server layers are: routes and middleware, controllers, services (OTP codes, export, mailer), utilities (validator, serializers, dates), and the MySQL connection pool.

3.2 Architectural Views

View	What it shows	Figure
Logical	Classes and packages of the frontend and backend and how they depend on each other	Figure 2 (D2)
Process	Runtime processes (Flutter app, Node.js server, MySQL, SMTP) and the message flows for registration, login, add transaction and export	Figures 3 to 7 (D3 to D7)
Physical	Hardware nodes and network links: user device, application server, database server, email provider	Figure 8 (D8)
Deployment	Artifacts placed on each node, such as the app build, server.js, .env and schema.sql	Figure 9 (D9)

Insert each figure under its row. Member A drew these views.

3.3 Technology Stack

Layer	Technology
Frontend	Flutter 3.27.4, Dart 3.6.2, provider, http, fl_chart, shared_preferences, file_picker, google_fonts, intl

Layer	Technology
Backend	Node.js 18 or later, Express 4, mysql2, jsonwebtoken, bcryptjs, nodemailer, pdfkit, helmet, cors, morgan, dotenv
Database	MySQL 8.0, InnoDB, utf8mb4

3.4 Coding Standards

- Dart code follows the lint rules in analysis_options.yaml and is checked with flutter analyze.
- Dart files use snake_case names, classes use UpperCamelCase, members use lowerCamelCase.
- JavaScript uses camelCase, one controller and one route file per resource, and async/await with asyncHandler for every route.
- All request input is validated through the Validator class before any database work.
- Errors use one JSON shape with message and code; secrets stay in the .env file.
- Design values (colours, spacing, shadows) are defined once in constants.dart.

3.5 Tools

Purpose	Tool
Editor and run	VS Code or Android Studio, flutter run, npm run dev
Version control	Git
API testing	Postman
Database	MySQL Workbench or the MySQL command line
Diagrams	PlantUML
Documents	Word processor or Claude Docs

3.6 Timeline

Phase	Planned dates	Main output
1. Requirements	3 Aug to 11 Aug 2026	SRS and use case diagram
2. Architecture and planning	12 Aug to 24 Aug 2026	Architecture views, standards, Gantt chart
3. Coding and unit testing	25 Aug to 10 Sep 2026	Source code and unit tests
4. Integration and system testing	11 Sep to 18 Sep 2026	Test cases, bug report, metrics, final report

Insert Figure 10 here: Gantt chart (diagram D13). Adjust these dates if your actual dates differed.

4. Phase 3: Coding and Unit Testing

4.1 Module Division

Both members divided the system into nine modules (Figure 11, diagram D10). Member A built the backend and Member B built the frontend.

Module	Content	Owner
M1 UI	22 pages and 6 reusable components in lib/pages and lib/components	Member B
M2 State and logic	ExpenseState, AuthService, CurrencyService	Member B
M3 API client	ApiService, the single HTTP client for all endpoints	Member B
M4 Models and utilities	User, FinancialTransaction, TransactionCategory, Budget, Account; constants, helpers, country and currency data	Member B

Module	Content	Owner
M5 Routing and middleware	Route files, requireAuth, error handler	Member A
M6 Controllers	Auth, user, transaction, category, budget, account, analytics	Member A
M7 Services	OTP codes, CSV and PDF export, mailer	Member A
M8 Backend utilities	Validator, serializers, dates, http, otp	Member A
M9 Database	Connection pool and schema.sql with six tables	Member A

4.2 Database Design

The database expensemate_db has six tables: users, verification_codes, categories, accounts, transactions and budgets (Figure 12, diagram D11). Foreign keys cascade on user deletion, while accounts and categories used by transactions are protected with RESTRICT. Amounts use DECIMAL(14,2), and unique keys prevent duplicate category names, account names and budget rows.

4.3 Key Implementation Features

Feature	How it works
Authentication	bcrypt password hashing; JWT issued on login or OTP verification; requireAuth loads the user on every protected request
OTP verification	6-digit code stored as a SHA-256 hash, 10-minute expiry, 5 attempts, 60-second resend cooldown
Budget alerts	Spent amount is summed per category and month; the bar is green below 85 percent, amber from 85 to 100 and red above 100 (Figure 13, diagram D12)
Budget carry-forward	A limit set in an earlier month applies until a later month overrides it

Feature	How it works
Analytics	Backend groups transactions into 8 three-hour blocks, 7 days, 4 to 5 weeks or 12 months; the app draws trend and donut charts with fl_chart
Export	CSV and PDF built on the server (pdfkit for PDF) and emailed as attachments
Currency conversion	Optional multiplication of balances, transactions and budget limits by a rate inside one database transaction
Offline-friendly caching	ExpenseState shows SharedPreferences data first, then syncs with the API when data is older than 15 minutes
Receipts	Camera or gallery image is base64-encoded and stored with the transaction

4.4 Unit Testing

Each member tested their own modules. Backend unit tests (67) cover the validator, date utilities, OTP service, auth middleware, error handler, serializers and controllers. Frontend unit tests (46) cover models, ExpenseState, services, utilities and reusable widgets. The test cases are listed in the Test Cases Document.

Level	Cases planned	Cases passed	Report
Backend unit	67	[fill]	Test Cases Document, section 3
Frontend unit	46	[fill]	Test Cases Document, section 4

5. Phase 4: Integration and System Testing

5.1 Integration Testing (Member A)

The modules were integrated in this order: database and schema, backend API, ApiService and ExpenseState, then the UI pages. The 40 integration test cases check authentication and OTP, transactions with account balances, budgets with carry-forward,

analytics, currency conversion with rollback, export emails, and the cache and sync behaviour between the app and the API.

5.2 System Testing (Member B)

The 35 system test cases run complete user journeys on a clean database: sign up and verification, daily transaction entry, budget alert colours, analytics periods, account and category management, profile settings, export, and logout and login. They also cover security (hashed storage, SQL injection text, token expiry), performance, compatibility, usability and reliability.

5.3 Code Complexity (Member A)

Cyclomatic complexity was measured for the controllers, services, utilities and main frontend files. A function with a CC above 10 was reviewed, and one above 20 was refactored. The highest values, averages and risk ratings are in the Metrics Report, section 4. Summary: [fill: highest CC, its function, and the average CC].

5.4 Fault Find and Fix Tracking (Member B)

Every defect was logged with an ID, module, severity and status, and was re-tested after fixing. The find rate is faults found per executed test case, and the fix rate is faults fixed divided by faults found. Results are in the Metrics Report, section 5. Summary: [fill: faults found, faults fixed, fix rate, open faults].

6. Challenges and Solutions

Challenge	Solution
Slow screens while waiting for the server	ExpenseState renders cached data from SharedPreferences first and refreshes in the background after 15 minutes
Budgets that should continue into new months	A limit applies to later months until a newer entry overrides it, so users set a budget once
Keeping amounts consistent when the currency changes	Balances, transactions and budget limits are converted inside a single database transaction

Challenge	Solution
Protecting verification codes	Only SHA-256 hashes are stored, with expiry, an attempt limit and a resend cooldown
Revealing which emails are registered	The forgot-password endpoint always returns the same message
Deleting data that other records depend on	RESTRICT foreign keys block the delete and the error handler returns a clear 409
Letting users restyle the whole app	AppColors.setPrimary updates the primary colours in one place and every widget reads from it
Receipts as images	The app encodes the image in base64 and the server accepts request bodies up to 10 MB

7. Maintenance Scope and Future Work

7.1 Maintenance Types

Type	Example for ExpenseMate
Corrective	Fix defects found in the bug report, such as a wrong budget percentage
Adaptive	Update Flutter, Node.js or MySQL versions; change the API base URL when the server moves
Perfective	Add new chart types or filters requested by users
Preventive	Refactor functions with high cyclomatic complexity; extend unit tests

The layered design limits the effect of a change: a new endpoint needs a route, a controller and a matching ApiService method, and UI changes stay in the pages and components.

7.2 Planned Enhancements

- Multi-currency support with live exchange rates; version 1.0 converts using a rate entered by the user.
- Cloud backup of user data.
- CSV import to match the existing CSV export.
- Push or email warnings when a budget reaches its limit.
- Storing receipts as files instead of base64 text in the database.
- Login rate limiting and automated tests in a continuous integration pipeline.

8. Conclusion

ExpenseMate meets the objectives of the assignment: it records income and expenses, enforces monthly budgets with colour-coded alerts, shows category-wise analytics, exports CSV and PDF reports, and was built and verified through requirements, architecture, construction, unit, integration and system testing. The Client-Server design with a layered backend keeps the code modular and ready for the enhancements listed in section 7. Working as a two-member team, the project also practised dividing a system into modules, planning with a Gantt chart, and measuring quality with complexity and fault metrics.

9. Appendices

9.1 Deliverables

Phase	Deliverable	Document
1	2-page SRS	SRS Document
2	Architecture document and Gantt chart	Sections 3.1 to 3.6 and Figures 2 to 10 of this report
3	Source code and unit test reports	Project repository; Test Cases Document
4	Test cases, bug report, complexity metrics	Test Cases Document; Metrics Report

9.2 Figures to Insert

Figure	Diagram	Source file block
1	Use Case Diagram	D1
2	Logical View	D2
3	Process View, runtime processes	D3
4 to 7	Sequence diagrams: registration, login, add transaction, export	D4 to D7
8	Physical View	D8
9	Deployment View	D9
10	Gantt chart	D13
11	Module decomposition	D10
12	ER diagram	D11
13	Budget alert activity diagram	D12

Render each diagram from expensemate_diagrams.puml and paste the image at the matching figure line.

9.3 References

- Flutter documentation: https://docs.flutter.dev
- Express documentation: https://expressjs.com
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- PlantUML documentation: https://plantuml.com